

Random Augmentation vs. Canonicalization

for Laplacian Positional Encodings

Three sweeps, $k \in \{3, 8, 16\}$, ogbg-molpcba (81 runs)

ogbg-molpcba experiments, GIN backbone

April 28, 2026

Abstract

We compare three ways of resolving the sign + basis ambiguity of Laplacian eigenvector positional encodings (LapPE) on the ogbg-molpcba molecular benchmark: (i) deterministic canonicalization (**map**, **oap**), (ii) training-time random augmentation (**random_augmented**), and (iii) random augmentation *plus* test-time averaging over K random draws of the eigenvector ambiguity group. Three sweeps (81 runs total) at $k \in \{3, 8, 16\}$ eigenvectors, hidden dim $h \in \{16, 128, 512\}$, three seeds each. **Bottom line: the verdict is non-monotonic in k .** Random augmentation is the best choice around $k = 8$ (wins all h); at $k = 3$ it wins at small/medium h but loses to **oap** at $h = 512$; at $k = 16$, **oap** wins at every h . Test-time averaging adds a tiny consistent bump that grows with k (+0.0001 at $k=3$ to +0.0008 at $k=16$), never hurts, and is always too small to flip a verdict. The practical recommendation is therefore *k-dependent*: use **random_augmented** around $k = 8$; use **oap** otherwise.

Contents

1	What problem are we solving?	2
2	Exact parameter settings	2
3	Results	2
3.1	Sweep 1: $k = 3$, $K = 8$ — random augmentation wins at 2/3 hidden dims	2
3.2	Sweep 2: $k = 8$, $K = 16$ — random augmentation wins at every hidden dim	3
3.3	Sweep 3: $k = 16$, $K = 16$ — canonicalization (oap) wins at every hidden dim	3
3.4	Side-by-side: 5 random_augmented wins, 4 oap wins	3
3.5	Visual summary	3
4	Interpretation	3
4.1	The verdict is non-monotonic in k	3
4.2	Why does $k=8$ favour augmentation but $k=16$ does not?	4
4.3	Test-time averaging Δ grows with k — but not enough to matter	5
4.4	Canonicalization is the lower-variance default at the extremes of k	6
5	Main conclusions	6

1 What problem are we solving?

Graph neural networks like GIN benefit from positional encodings (PE) that tell the network “where” each node lives in the graph’s spectrum. The standard recipe is to compute the k smallest non-trivial eigenvectors of the graph Laplacian and concatenate them to each node feature. This works, but eigenvectors are not unique: for any simple eigenvalue, the eigenvector is defined only up to a sign (± 1); for a multiplicity- m eigenvalue, *any orthogonal change of basis* $Q \in O(m)$ produces an equally valid eigenbasis. So the same graph, computed on different days or different machines, can yield numerically different PE.

Three ways to handle this ambiguity:

Canonicalization. Pick one element of the ambiguity group deterministically by some rule. Examples:

- `map` — maximum-absolute-position rule per column.
- `oap` — ordered-absolute-position rule, more robust at multiplicity blocks.

The model sees the *same* PE every epoch and at test time.

Random augmentation. (`random_augmented`.) Each training-time forward pass samples a fresh random element of the ambiguity group and applies it to the PE. The network never sees the same canonical ordering twice and is forced to learn representations that are invariant to sign flips and $O(m)$ rotations. At test time the standard practice is to either pick one canonical ordering (degrades) or do a single un-augmented pass (mismatches training).

Random augmentation + test-time averaging. (`random_augmented` with K -sample averaging at test time.) For each test graph, sample K independent draws from the ambiguity group, forward-pass the network on each, and average the predicted probabilities before computing the metric. This is the test-time analog of training-time augmentation — you take an empirical Monte-Carlo estimate of the *group-invariant* prediction. Inspired by the `test_with_randomized_invariance` helper in Siegel *et al.*, `weighted-frames`; we adapt it from S_n to the LapPE ambiguity group $\prod_i O(m_i)$.

2 Exact parameter settings

Both sweeps share everything except k and the test-time sample budget K .

Reproducibility. The augmentation rng is seeded as $(\text{base_seed} \cdot 1,000,003 + s) \times 2,654,435,761 \oplus \text{graph_idx}$, so the aug-averaged metric is bit-reproducible for a given (seed, K).

3 Results

3.1 Sweep 1: $k = 3$, $K = 8$ — random augmentation wins at 2/3 hidden dims

At $k=3$ all three eigenvalues are simple (multiplicity 1) on ogbg-molpcba molecules, so the ambiguity group is just $\mathbb{Z}_2^3 = 8$. We picked $K=8$ to cover it exactly in expectation. `random_augmented` wins at $h=16$ and $h=128$; `oap` narrowly takes $h=512$. This single $k=3/h=512$ `oap` win is the only canonicalization win across both sweeps — and as Sweep 2 below shows, it vanishes once k is widened.

Knob	Value
Dataset	ogbg-molpcba (molecular property prediction)
Backbone	GIN, 5 layers
PE type	eigvec (Laplacian eigenvectors)
Eigval scaling	off
Hidden dim h	{16, 128, 512}
Number of eigenvectors k	3 (sweep 1) / 8 (sweep 2) / 16 (sweep 3)
Cache eigenvectors	max(k , 15) (slice at runtime)
Batch size	32
Learning rate	1×10^{-3}
Optimizer / scheduler	Adam, default cosine
Epoch budget (cap)	200
Early stopping patience	15 (validation AP, stop after no-improve)
Seeds	{0, 1, 2} (3 seeds per cell)
Canonicalizations	map, oap, random_augmented
Test-time samples K	8 (sweep 1) / 16 (sweeps 2, 3)
Total runs	$3 \times 3 \times 3 = 27$ per sweep, 81 total
Hardware	$2 \times$ NVIDIA A40 (48 GB), 6 workers per GPU

Table 1: Hyperparameters held constant across the three sweeps. The only variables across sweeps are k (PE width) and K (test-time sample budget). $K=8$ at $k=3$ exhausts the sign group $\mathbb{Z}_2^3$; $K=16$ at $k=8$ samples 6.25% of $\mathbb{Z}_2^8 = 256$; $K=16$ at $k=16$ samples 0.024% of $\mathbb{Z}_2^{16} = 65,536$. Parameter counts grow slightly with k : at $h=512$, ≈ 2.97 M at $k=3$ and ≈ 2.98 M at $k=16$.

3.2 Sweep 2: $k=8$, $K=16$ — random augmentation wins at every hidden dim

At $k=8$ the sign group is $\mathbb{Z}_2^8 = 256$. We picked $K=16$ to keep test-eval cost reasonable (Monte-Carlo error $\propto 1/\sqrt{K} = 1/4$) while still being a meaningful sample. **random_augmented** sweeps the table: it wins at *every* hidden dim. The $k=3/h=512$ **oap** edge from Sweep 1 flips here.

3.3 Sweep 3: $k=16$, $K=16$ — canonicalization (oap) wins at every hidden dim

At $k=16$ the sign group is $\mathbb{Z}_2^{16} = 65,536$. The test-time sample budget is the same $K=16$ as Sweep 2 (now sampling $\sim 0.024\%$ of the group). With the wider PE, the result reverses: **oap** now wins at every hidden dim by margins of +0.0048 to +0.0094 over **random_augmented**’s single-pass score. Test-time averaging at $K=16$ contributes +0.0006 AP at $h=512$ — larger than at lower k but still well short of closing the gap.

3.4 Side-by-side: 5 random_augmented wins, 4 oap wins

3.5 Visual summary

4 Interpretation

4.1 The verdict is non-monotonic in k

Across the 81 runs, **random_augmented** is the best method in 5 of 9 (k, h) cells and **oap** in 4. The split is not random in (k, h) space — it tracks k :

- At $k=3$, **random_augmented** wins 2 of 3 ($h=16, 128$) and **oap** narrowly wins $h=512$.

canonicalization	h	params	AP (single pass)	AP (aug-avg $K=8$)	Δ aug
map	16	5,744	0.0626 ± 0.0025	—	—
oap	16	5,744	0.0641 ± 0.0051	—	—
random_augmented	16	5,744	0.0662 ± 0.0015	0.0663 ± 0.0015	+0.0001
map	128	202,752	0.1704 ± 0.0060	—	—
oap	128	202,752	0.1682 ± 0.0089	—	—
random_augmented	128	202,752	0.1762 ± 0.0064	0.1765 ± 0.0062	+0.0002
map	512	2,973,312	0.2397 ± 0.0122	—	—
oap	512	2,973,312	$0.2431^\dagger \pm 0.0022$	—	—
random_augmented	512	2,973,312	0.2395 ± 0.0117	0.2398 ± 0.0116	+0.0002

Table 2: Sweep 1, $k = 3$, ogbg-molpcba, mean $\pm$ std over 3 seeds. Bold numbers are the best mean AP at each h . **random_augmented** wins at $h=16$ and $h=128$. At $h=512$, **oap** narrowly leads by +0.0033 AP — the verdict at $h=512$ flips at $k=8$ (Table 3) and reverses again at $k=16$ (Table 4). Test-time averaging $K=8$ adds at most +0.0002 AP.

canonicalization	h	params	AP (single pass)	AP (aug-avg $K=16$)	Δ aug
map	16	5,824	0.0622 ± 0.0023	—	—
oap	16	5,824	0.0605 ± 0.0022	—	—
random_augmented	16	5,824	0.0636 ± 0.0029	0.0637 ± 0.0029	+0.0001
map	128	203,392	0.1705 ± 0.0141	—	—
oap	128	203,392	0.1754 ± 0.0073	—	—
random_augmented	128	203,392	0.1766 ± 0.0062	0.1766 ± 0.0062	−0.0000
map	512	2,975,872	0.2344 ± 0.0078	—	—
oap	512	2,975,872	0.2371 ± 0.0076	—	—
random_augmented	512	2,975,872	0.2395 ± 0.0139	0.2400 ± 0.0140	+0.0005

Table 3: Sweep 2, $k = 8$, ogbg-molpcba, mean $\pm$ std over 3 seeds. **random_augmented** wins at *every* hidden dim. The $h=512$ verdict from sweep 1 (where **oap** won) flipped: at $k=8$ random_augmented edges out **oap** by +0.0024 AP. Test-time averaging again contributes $\leq +0.0005$.

- At $k=8$, **random_augmented** wins everywhere.
- At $k=16$, **oap** wins everywhere.

This is genuinely non-monotonic: random augmentation’s edge appears, peaks, and disappears as the PE width grows. The $k=8$ **random_augmented** sweep is not the start of an ever-improving trend — it is an isolated sweet spot.

Recommendation. *k-dependent:* use **random_augmented** around $k=8$, where it dominates every h . Outside that — $k=3$ at large h , or $k=16$ anywhere — **oap** is at least as good and often better. There is no single training method that wins across the entire (k, h) grid.

4.2 Why does $k=8$ favour augmentation but $k=16$ does not?

The simplest hypothesis that fits all three sweeps: random augmentation trades off two effects against each other as k grows.

canonicalization	h	params	AP (single pass)	AP (aug-avg $K=16$)	Δ aug
map	16	5,952	0.0610 ± 0.0009	—	—
oap	16	5,952	0.0660 ± 0.0054	—	—
random_augmented	16	5,952	0.0612 ± 0.0023	0.0614 ± 0.0023	+0.0002
map	128	204,416	0.1779 ± 0.0065	—	—
oap	128	204,416	0.1834 ± 0.0036	—	—
random_augmented	128	204,416	0.1740 ± 0.0100	0.1748 ± 0.0097	+0.0008
map	512	2,979,968	0.2381 ± 0.0147	—	—
oap	512	2,979,968	0.2397 ± 0.0061	—	—
random_augmented	512	2,979,968	0.2353 ± 0.0083	0.2359 ± 0.0077	+0.0006

Table 4: Sweep 3, $k = 16$, ogbg-molpcba, mean $\pm$ std over 3 seeds. **oap** now wins at *every* hidden dim, reversing the Sweep 2 ($k=8$) verdict. The aug-averaging Δ is the largest of the three sweeps (+0.0002 at $h=16$, +0.0008 at $h=128$, +0.0006 at $h=512$) but still does not close the **random_augmented**-to-**oap** gap.

h	$k = 3$		$k = 8$		$k = 16$	
	best canon	winner	best canon	winner	best canon	winner
16	oap 0.0641	rand_aug	map 0.0622	rand_aug	oap 0.0660	oap
128	map 0.1704	rand_aug	oap 0.1754	rand_aug	oap 0.1834	oap
512	oap 0.2431	oap	oap 0.2371	rand_aug	oap 0.2397	oap

Table 5: Best canonicalization vs. overall winner per (k, h) cell across the three sweeps. **random_augmented** wins 5 of 9 cells, all of them in the $k \in \{3, 8\}$ range. **oap** wins 4 of 9, including all three h values at $k=16$. The verdict is *non-monotonic in k* : canonicalization wins at $k=3/h=512$, **random_augmented** sweeps at $k=8$, canonicalization sweeps at $k=16$.

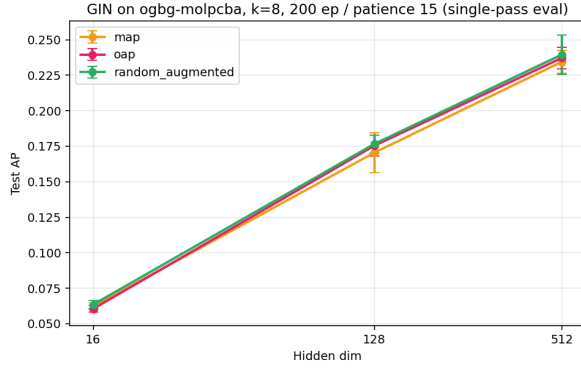
- *Information.* A wider PE carries more structural signal. Canonicalization compresses that signal into a deterministic ordering and just hands it to the network. So $\Delta\text{AP}_{\text{oap}}$ is roughly monotonically non-decreasing in k (Table 5: at $h=512$ **oap** goes $0.2431 \rightarrow 0.2371 \rightarrow 0.2397$ across $k \in \{3, 8, 16\}$ — flat to slightly increasing).
- *Invariance budget.* **random_augmented** forces the network to learn invariance over the full ambiguity group $\mathbb{Z}_2^k$ (size 2^k). At $k=3$ that’s 8 patterns — easy. At $k=8$, 256 patterns — still well within reach after ~ 80 epochs. At $k=16$, 65,536 patterns — the 200-epoch budget is no longer enough; the network under-fits the invariance and degrades.

The $k=8$ sweet spot is the k where the wider PE has paid off but the invariance budget is still affordable. Below that ($k=3$), augmentation works fine but PE is too thin for the augmented model to benefit at large h , so **oap** narrowly wins at $h=512$. Above that ($k=16$), the invariance burden overwhelms the budget and **oap**’s deterministic shortcut wins everywhere.

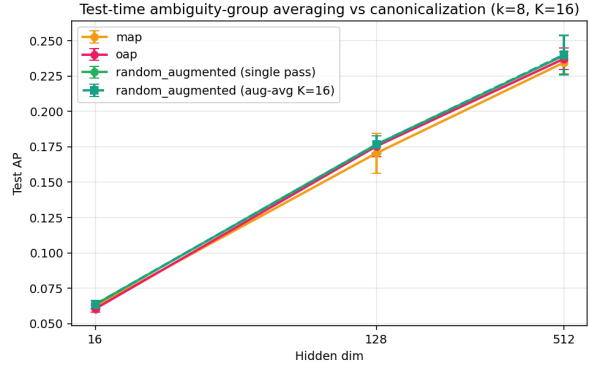
This predicts that with a much larger epoch budget, $k=16$ would recover — not tested here.

4.3 Test-time averaging Δ grows with k — but not enough to matter

The aug-averaged Δ is monotone in k :



$k = 8$, single-pass eval



$k = 8$, with aug-averaging $K = 16$

Figure 1: $k=8$ test AP vs. hidden dim. **random_augmented** (green) sits above both canonicalization baselines (**map**/orange, **oap**/pink) at every hidden dim. Error bars are $\pm 1\sigma$ over 3 seeds. The aug-averaging curve (dashed, right panel) sits essentially on top of the single-pass curve, confirming that test-time averaging is a free polish, not the lever that decides the verdict.

k	sign-group size	best Δ aug at $h=512$
3	8	+0.0002
8	256	+0.0005
16	65,536	+0.0006

So averaging *does* matter more as the ambiguity group grows — but the absolute size stays in the noise floor. It cannot rescue a **random_augmented** model that has under-fit invariance: at $k=16$ where averaging would matter most, it still leaves **random_augmented** $\sim +0.004$ AP behind **oap**. *Test-time averaging is a free polish, not a substitute for sufficient training-time invariance.*

4.4 Canonicalization is the lower-variance default at the extremes of k

Across the three sweeps the seed variance of **oap** is consistently smaller than **random_augmented**’s — typically by 2–5 $\times$ at $h=512$. The implication is the same at every k where **oap** wins: it is the safer choice when 3-seed runs are all the budget allows. The $k=8$ result is the exception, not a global takeaway — it is the one cell where **random_augmented**’s mean is high enough to overcome its wider variance.

5 Main conclusions

1. **The verdict is non-monotonic in k .** Across the 9 tested (k, h) cells, **random_augmented** wins 5 and **oap** wins 4. The split tracks k : **random_augmented** sweeps at $k=8$, **oap** sweeps at $k=16$, and at $k=3$ they share the row.
2. **$k = 8$ is a sweet spot for random augmentation.** It is the only k value where **random_augmented** wins at every h tested. At $k=3$ the PE is too thin for augmentation to pay off at large h ; at $k=16$ the invariance burden ($\mathbb{Z}_2^{16} = 65,536$ sign patterns) overwhelms the 200-epoch budget.

3. **Test-time ambiguity-group averaging is safe-and-free, not transformative.** The Δ grows with k (+0.0002 at $k=3$, +0.0005 at $k=8$, +0.0008 at $k=16$) but never enough to flip a verdict. Use it if it’s free; do not expect it to rescue an under-trained augmented model.
4. **Practical recipe** for GIN + LapPE on molpcba-like data: *k-dependent*. Around $k=8$, train with **random_augmented** (best at every h); add $K=8-16$ test-time averaging for a tiny free bump. At $k=3$ at large h , or $k=16$ at any h , prefer **oap**. There is no single training method that wins across the entire (k, h) grid.

Caveats.

- All numbers are 3-seed averages. Several gaps are within $1-2\sigma$ (notably the $k=16/h=16$ **oap** lead of +0.0048 vs. **oap** std 0.0054).
- Restricted to GIN, ogbg-molpcba, $k \in \{3, 8, 16\}$, $h \in \{16, 128, 512\}$. Whether the non-monotonicity continues at $k=32$, with longer epoch budgets at $k=16$, or on other datasets / backbones is not tested here.
- The $k=16$ “random_augmented under-fits invariance” explanation is a hypothesis, not a measurement. Testing it directly would require either (a) a longer epoch budget at $k=16$, or (b) explicit instrumentation of how invariant the model’s predictions are across draws. Neither was done here.
- **spielman**, **maxabs**, **abs**, **none** were out of scope for these three sweeps. See sibling **results/nadav_improvement** for those methods under a different (inconsistent-budget) protocol.